Atrosha: The Definitive Engineering Book

ACode-First Guide to the Architecture and Mathematics

Note: This book documents the full engineering surface area of Atrosha. It uses clear, direct language and maps every theoretical concept directly to the corresponding source code. Analogies and metaphors are intentionally omitted for conciseness and precision.

CHAPTER 1: The Three-Tier Architecture

To securely process non-deterministic AI actions, Atrosha strictly separates state management across three database engines based entirely on latency bounds.

1.1 The Hot Tier: Redis (Latency < 1ms)

Redis is utilized exclusively for synchronous request interception. It runs inside the RAM and handles counters that must be decremented on the critical network path.

- **Token Bucket Rate Limiting:** We implement rate limiting using the key `rate:{org_id}:{agent_id}:{minute}`. A Lua script executes atomically to ensure the Python agent does not exceed requests-per-minute (RPM) limits.
- **Budget Accumulators:** `atrosha:{org_id}:budget:{agent_id}`. The Rust proxy runs an atomic `INCRBY` command during a transaction attempt. If the accumulation exceeds the daily limit, the request returns HTTP 403.
- **Circuit Breaker:** `cb:{agent_id}:deny_count`. If a high frequency of consecutive HTTP 403s occurs, the `open` key flag is set. The proxy drops subsequent traffic dynamically before it hits the backend logic.

1.2 The Warm Tier: PostgreSQL via Supabase (Latency < 10ms)

PostgreSQL handles standard relational states with Row Level Security (RLS) ensuring strict multi-tenant data isolation.

Schema Design Highlights:

```
CREATE TABLE agents (
    id UUID PRIMARY KEY,
    organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    pubkey TEXT NOT NULL, -- The Ed25519 Public Key for agent validation
    daily_limit_cents BIGINT NOT NULL
);
```

pgvector Embeddings:
The `rule_embeddings` table stores `vector(384)` configurations. Human guidelines (e.g., "Do not pay independent contractors") are embedded using `MiniLM-L6-v2`.
When the agent trace executes, we compute the strict Cosine Similarity S_C:
$ S_C(A, B) = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| |\mathbf{B}|} $

1.3 The Cold Tier: ClickHouse (Latency < 100ms)

ClickHouse stores the immutable transaction ledgers for auditing. The Rust `AuditLogger` offloads database writes to asynchronous `tokio::spawn` threads so the proxy never blocks to perform intensive disk I/O.
We utilize the `MergeTree` engine partitioned by `toYYYYMMDD(ts)` to query billions of interactions sequentially without CPU saturation.

CHAPTER 2: The Rust Proxy Engine (`proxy/src/`)

The proxy enforces all rules. It is written in Rust to guarantee memory safety—specifically preventing buffer overflows and arbitrary memory referencing.

2.1 The Global State (`proxy/src/state.rs`)

Atrosha relies on zero-cost abstractions to pass states down the HTTP pipeline. The `AppState` struct is cloned across requests using an `Arc` (Atomic Reference Count).
It holds:

1. `PolicyEngine`, `AgentRegistry`, `PermitValidator`
2. `redis_client`
3. Network endpoints: `HttpAdapter`, `EvmAdapter`, `AchAdapter`.
4. Threat interfaces: `SemanticClient`, `EgressWhitelist`.
5. Cryptography structures: `zkp_setup`, `poseidon_config`, `state_oracle`.

2.2 The Semantic Firewall (`proxy/src/semantic.rs`)

The `SemanticClient` interfaces with our localized Python FastAPI instance using the `reqwest` HTTP library. It is configured with a strict 500ms timeout so the proxy never degrades under high inference load.

The client hits three main endpoints:

- **`POST /classify`:** Sends the URL, Header elements, and JSON payload to the model. The model scores the payload and returns `SemanticVerdict` (`ALLOW`, `DENY`, or `QUARANTINE`) along with an operational confidence double.
- **`POST /verify`:** Matches the human `intent` against the proposed agent `action`. It enforces Cosine Similarity thresholds dynamically.
- **`POST /embed-fixed`:** Fetches 16-bit integer-quantized embeddings (`EmbedFixedResult`) so the vectors can be efficiently translated into finite fields for zkSNARK circuits.

2.3 The Verification Engine (`proxy/src/zkp/verifier.rs`)

The proxy utilizes the `ark_groth16` library to establish mathematical proofs that an agent adhered to the internal state configuration.
The `ProofVerifier` struct exposes an internal `/verify-proof` REST API that takes in a base64 string, converts it to an elliptic curve coordinate set, and processes mathematical execution in $O(1)$ constant time.

The verification key is cached statically globally via `OnceLock`.

```
pub fn verify_proof(vk: &VerifyingKey<Bn254>, proof: &Proof<Bn254>, public_inputs: &[Fr]) -> Result<bool, String>
```

The mandatory public inputs array injected natively against the proof include:

- `whitelist_root`: The compiled root hash of valid URLs.
- `request_body_hash`: The SHA identifier of the JSON transaction.
- `timestamp`: Temporal upper bound limit.
- `policy_version` & `state_seq`: Rollback mitigators ensuring the proof relies on the strict, real-time database state sequence.

CHAPTER 3: The Sovereign Agent (`sovereign_agent/`)

The `sovereign_agent/` isolates the stochastic capabilities of Large Language Models and Python scripting variables, separated completely from the Rust deterministic zone.

3.1 The ReAct Execution Loop (`loop.py` & `brain.py`)

Incoming requests traverse `server.py` into the core reasoning loop.
The LLM logic evaluates `tools.py` continuously until it determines the context is complete. No external port connection commands exist in the Python code surface. If the Python agent requires executing a database write or pushing to an external API, the connection defaults routing exclusively to `http://localhost:8080` (The Rust Proxy).

3.2 Deterministic Financial Logic (`payroll_engine.py`)

Neural networks evaluate float probabilities and are inherently unqualified to perform strict integer boundary tests reliably.
Therefore, `payroll_engine.py` implements a Moving Average algorithm statically for anomaly detection:
$ \mu_{3} = \frac{1}{3} \sum_{i=1}^3 S_{t-i} $
If standard deviation computations breach static boundary tolerances ($S_t > \mu_3 + (k \cdot \sigma)$), the Python codebase fires an exception that escalates the sequence out of the agent's hands before evaluating against the Rust proxy rulesets.

CHAPTER 4: The Theoretical Five Pillars

The underlying math executing the Atrosha framework.

4.1 Zero-Knowledge Proofs (zkSNARKs)

We process human policies into polynomials to define a Rank-1 Constraint System (R1CS):
$ A \cdot \vec{x} \circ B \cdot \vec{x} = C \cdot \vec{x} $
Using Groth16, these boundary equations map to the `ark_bn254` elliptic curves. This guarantees that policies evaluate deterministically without executing the policy code sequentially at runtime.

4.2 Causal Intent Graphs (do-calculus)

We process Agent reasoning files via Directed Acyclic Graphs (DAGs). We measure the mathematical isolation of human intent via Judea Pearl's intervention equations:
$ P(Y | do(X)) = \sum_z P(Y | X, Z) P(Z) $
If assessing the removal of the human intent yields statistically insignificant distribution shifts from the presence of the human intent ($P(Y \mid do(\neg X)) \approx P(Y \mid do(X))$), we mathematically confirm the LLM acted upon an injected external prompt or hallucinated, rather than obeying the input variable X.

4.3 Self-Evolving Adversarial Immune System

Red Team AI systems compute permutations against the input embedding vector to maximize internal loss J, searching for bypass arrays using the Fast Gradient Sign Method (FGSM):
$ x' = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y)) $
Blue Team models read x' arrays via ClickHouse logs and initiate background tasks inside `semantic_engine/train_all.py` to recursively close the gradient gap dynamically.

4.4 Hardware-Rooted Agent Identity (TEE)

The Python framework operates inside a Trusted Execution Environment (e.g., AWS Nitro). The physical silicon processes a $\text{Hash}_{\text{SHA384}}$ sum against the exact RAM state. During Ed25519 signature execution, this attestation document is sent. The `middleware.rs` validates the hardware signature against Intel/Amazon Root CAs to verify the Python framework processes haven't been dynamically manipulated via hypervisor exploits.

4.5 Temporal Logic Model Checking (LTL)

Unit tests lack coverage density. Atrosha transforms internal state machines into Kripke structures and constructs LTL algorithms:
$ \Box , (\neg q , \mathcal{U} , p) $
(It is Always True that property q will not occur Until property p is satisfied).
We translate to Büchi Automata. The pipeline solves the SAT configuration boundaries to mathematically prove the code array lacks exploitable states prior to deployment.